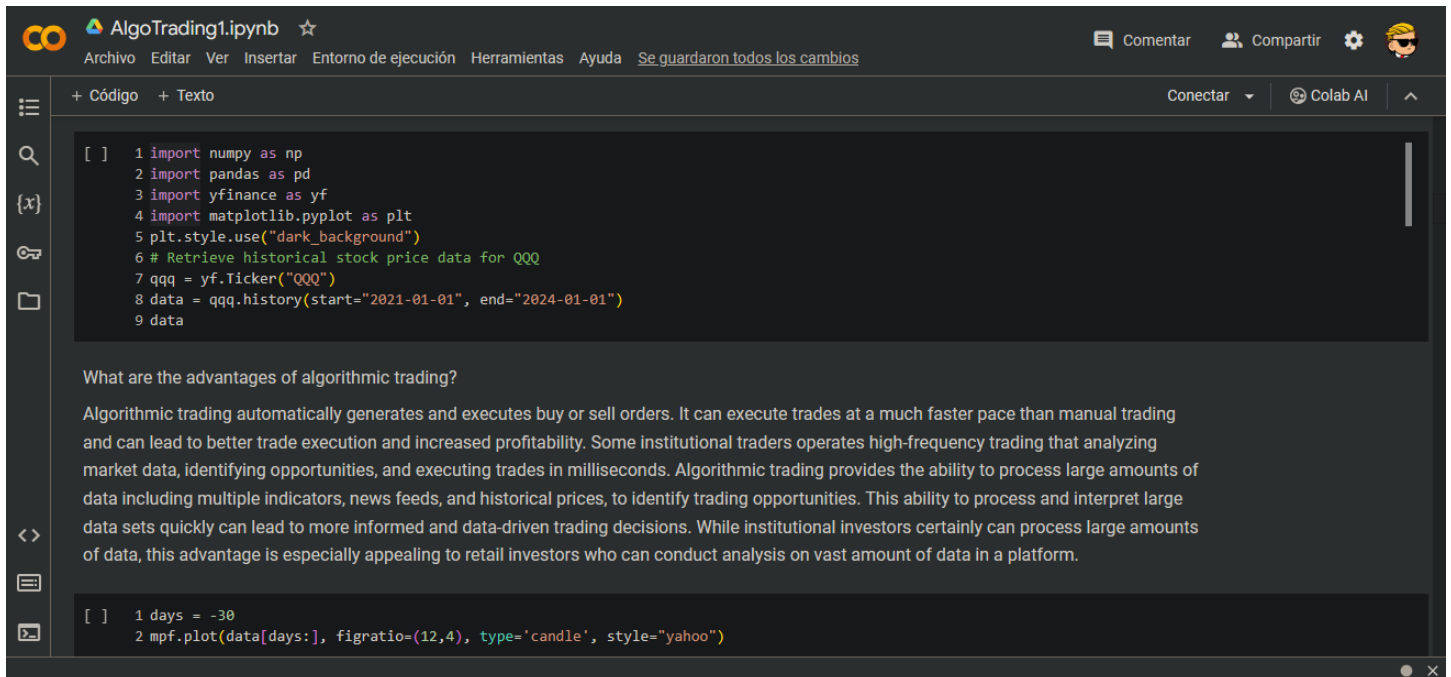


ALGORITHMIC TRADING USING TECHNICAL INDICATORS

AARON DE LA ROSA

Algorithmic trading is also called algo trading or automated trading. The algorithms are pre-researched rules that instruct when and how trades are executed based on various market data and conditions. These algorithms are viewed as the secret source of profitable trades. Algorithmic trading involves extensive data analysis for price movements, trading volume, technical indicators, and fundamental data.



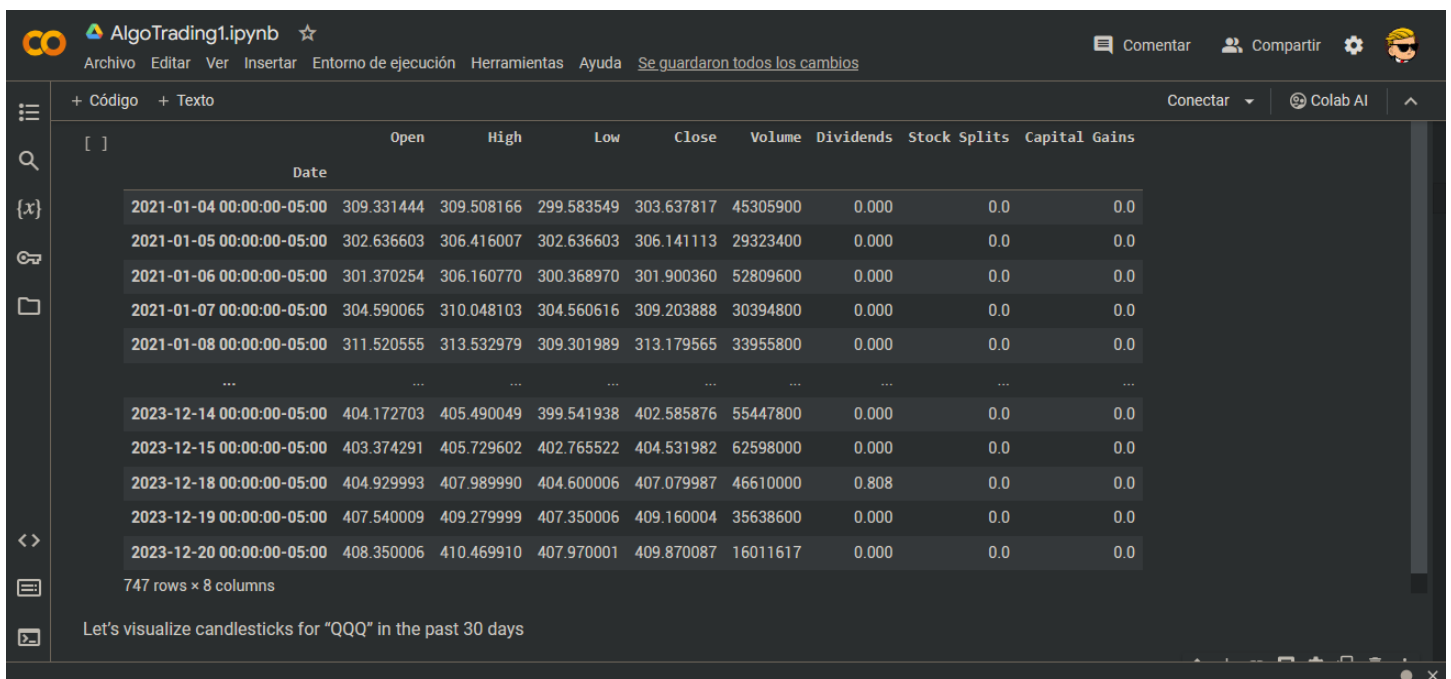
The screenshot shows a Jupyter Notebook interface with the title 'AlgoTrading1.ipynb'. The top bar includes navigation links like 'Archivo', 'Editar', 'Ver', 'Insertar', 'Entorno de ejecución', 'Herramientas', 'Ayuda', and a link to 'Se guardaron todos los cambios'. The left sidebar has icons for file explorer, search, and other functions. The main area contains a code cell with the following Python code:

```
[ ] 1 import numpy as np
2 import pandas as pd
3 import yfinance as yf
4 import matplotlib.pyplot as plt
5 plt.style.use("dark_background")
6 # Retrieve historical stock price data for QQQ
7 qq = yf.Ticker("QQQ")
8 data = qq.history(start="2021-01-01", end="2024-01-01")
9 data
```

Below the code cell is a text cell with the heading 'What are the advantages of algorithmic trading?'. The text explains that algorithmic trading automatically generates and executes buy or sell orders, leading to better trade execution and increased profitability. It mentions that some institutional traders use high-frequency trading to analyze market data, identify opportunities, and execute trades in milliseconds. It also notes that algorithmic trading provides the ability to process large amounts of data, including multiple indicators, news feeds, and historical prices, to identify trading opportunities. This ability to process and interpret large data sets quickly can lead to more informed and data-driven trading decisions. While institutional investors certainly can process large amounts of data, this advantage is especially appealing to retail investors who can conduct analysis on vast amounts of data in a platform.

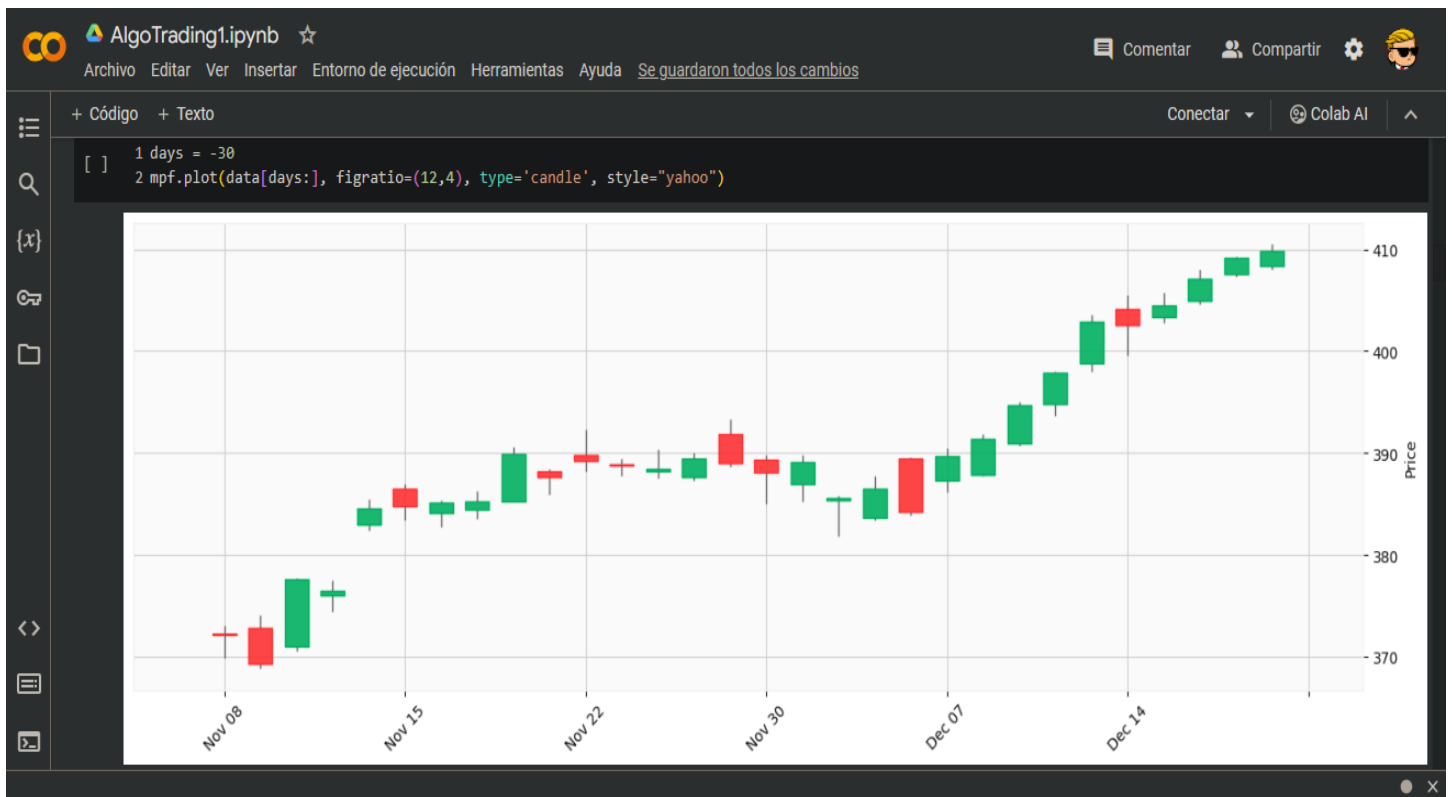
At the bottom, there is another code cell with the following code:

```
[ ] 1 days = -30
2 mpf.plot(data[days:], figratio=(12,4), type='candle', style="yahoo")
```



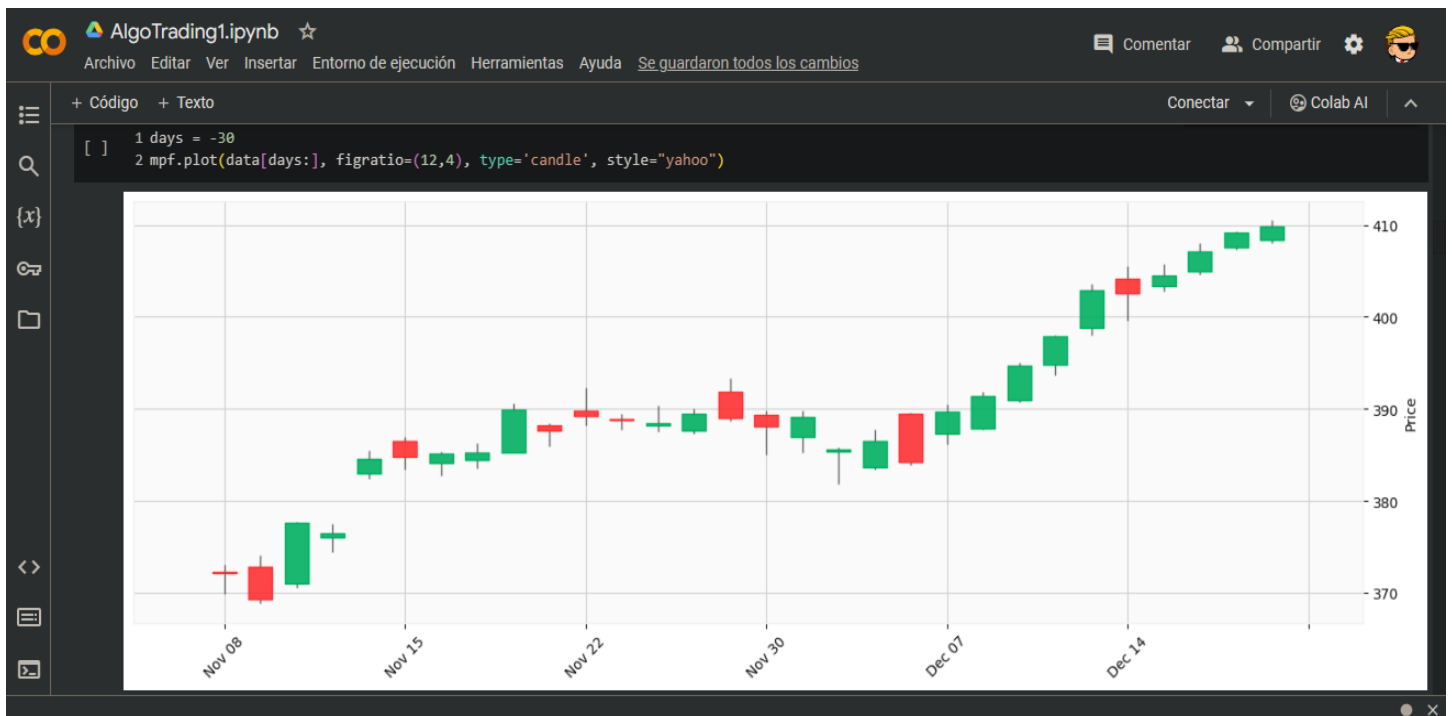
The screenshot shows the same Jupyter Notebook interface, but now displaying a table of historical stock data for QQQ. The table has 10 columns: 'Date', 'Open', 'High', 'Low', 'Close', 'Volume', 'Dividends', 'Stock Splits', and 'Capital Gains'. The data is presented in a dark-themed table with alternating row colors. The first few rows show data from 2021-01-04 to 2021-01-08, followed by an ellipsis indicating more rows, and then data from 2023-12-14 to 2023-12-20. The table is followed by a text cell that says '747 rows x 8 columns' and 'Let's visualize candlesticks for "QQQ" in the past 30 days'.

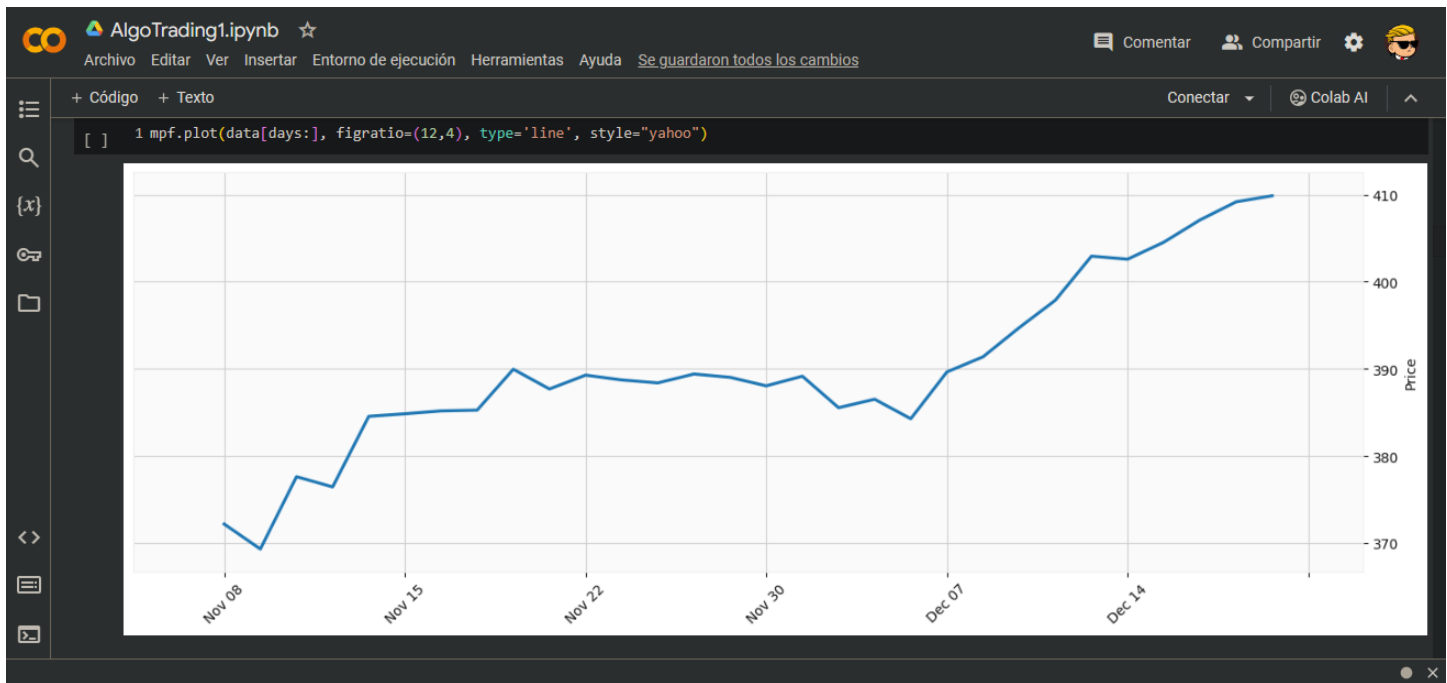
Date	Open	High	Low	Close	Volume	Dividends	Stock Splits	Capital Gains
2021-01-04 00:00:00-05:00	309.331444	309.508166	299.583549	303.637817	45305900	0.000	0.0	0.0
2021-01-05 00:00:00-05:00	302.636603	306.416007	302.636603	306.141113	29323400	0.000	0.0	0.0
2021-01-06 00:00:00-05:00	301.370254	306.160770	300.368970	301.900360	52809600	0.000	0.0	0.0
2021-01-07 00:00:00-05:00	304.590065	310.048103	304.560616	309.203888	30394800	0.000	0.0	0.0
2021-01-08 00:00:00-05:00	311.520555	313.532979	309.301989	313.179565	33955800	0.000	0.0	0.0
...	...	...	...	...	...	...	...	...
2023-12-14 00:00:00-05:00	404.172703	405.490049	399.541938	402.585876	55447800	0.000	0.0	0.0
2023-12-15 00:00:00-05:00	403.374291	405.729602	402.765522	404.531982	62598000	0.000	0.0	0.0
2023-12-18 00:00:00-05:00	404.929993	407.989990	404.600006	407.079987	46610000	0.808	0.0	0.0
2023-12-19 00:00:00-05:00	407.540009	409.279999	407.350006	409.160004	35638600	0.000	0.0	0.0
2023-12-20 00:00:00-05:00	408.350006	410.469910	407.970001	409.870087	16011617	0.000	0.0	0.0



What are the advantages of algorithmic trading?

Algorithmic trading automatically generates and executes buy or sell orders. It can execute trades at a much faster pace than manual trading and can lead to better trade execution and increased profitability. Some institutional traders operate high-frequency trading that analyzing market data, identifying opportunities, and executing trades in milliseconds. Algorithmic trading provides the ability to process large amounts of data including multiple indicators, news feeds, and historical prices, to identify trading opportunities. This ability to process and interpret large data sets quickly can lead to more informed and data-driven trading decisions. While institutional investors certainly can process large amounts of data, this advantage is especially appealing to retail investors who can conduct analysis on vast amount of data in a platform.





Renko chart

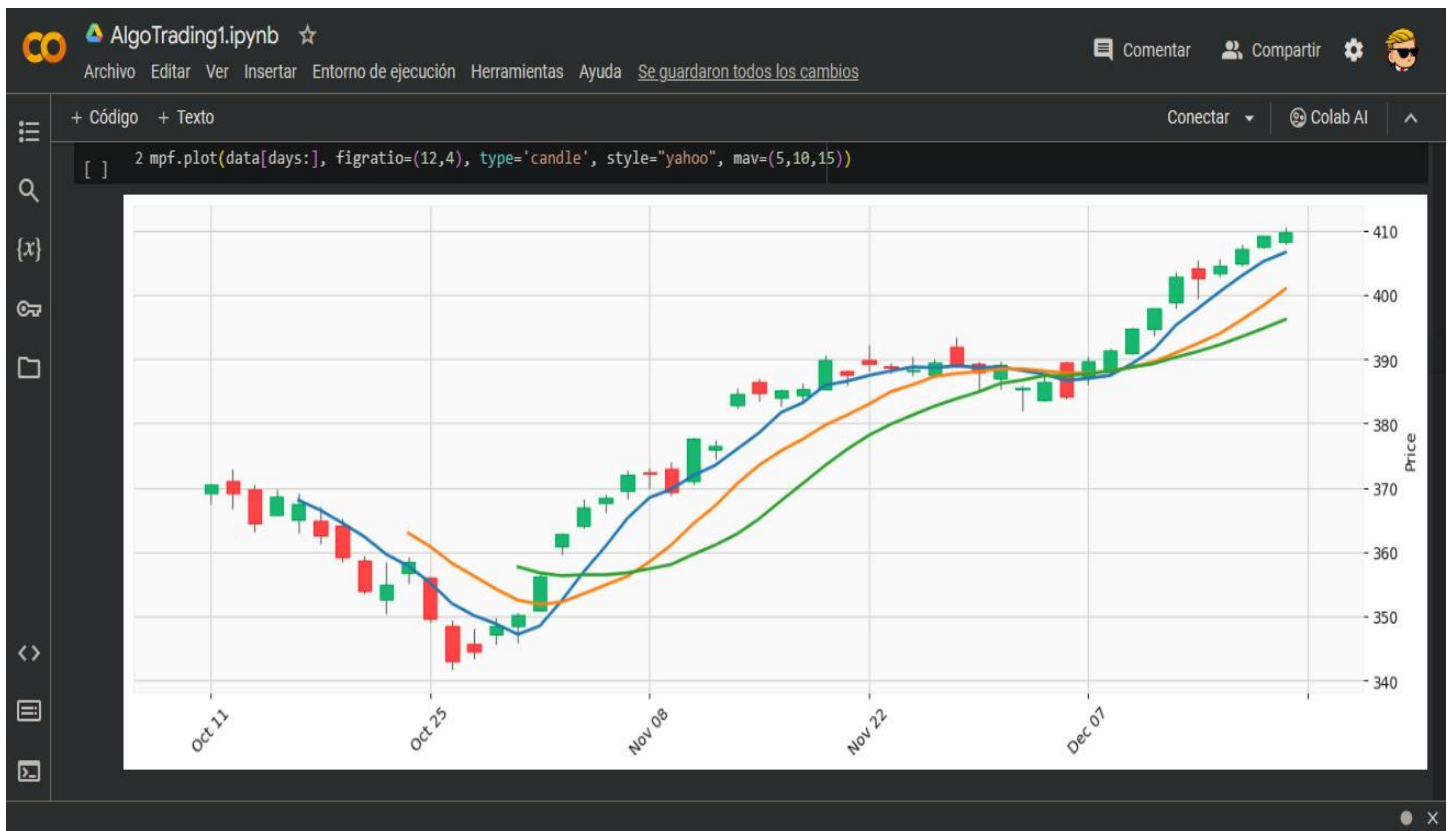
A Renko chart is a type of financial chart to visualize price movements in a simplified manner. It is named after the Japanese word “renga,” which means “brick,” as the chart consists of bricks or blocks that represent price movements. Unlike traditional price charts like line charts, bar charts, or candlestick charts that based on time intervals, Renko charts focus solely on price movement. Renko charts help filter out market noise and focus on the underlying price trend. Traders and technical analysts use Renko charts to identify trends, support and resistance levels, and potential entry or exit points for trading strategies.

Renko charts have two main types of bricks: bullish (typically colored white or green) and bearish (usually colored black or red). If the price moves upward and surpasses the top of the previous brick by at least the brick size, a new bullish brick is added above the previous one. Conversely, if the price moves downward and falls below the bottom of the previous brick by the brick size, a new bearish brick is added below the previous one. Let’s plot it now.

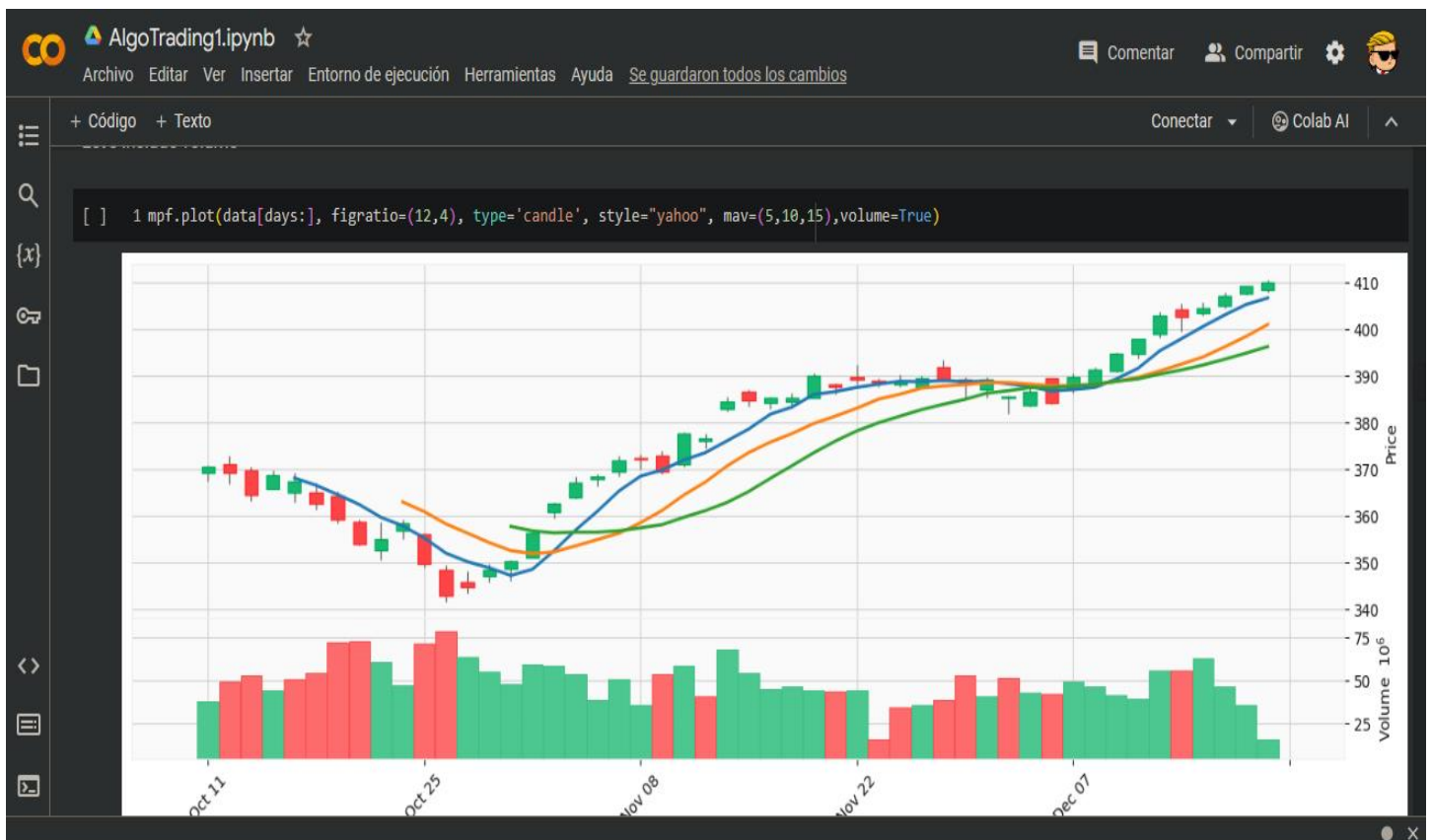


You can observe a long uptrend in recent days in the above Renko Chart. This helps traders to quick evaluate an uptrend or downtrend. Candle chart with moving average lines.

We add moving average lines to the candlesticks.

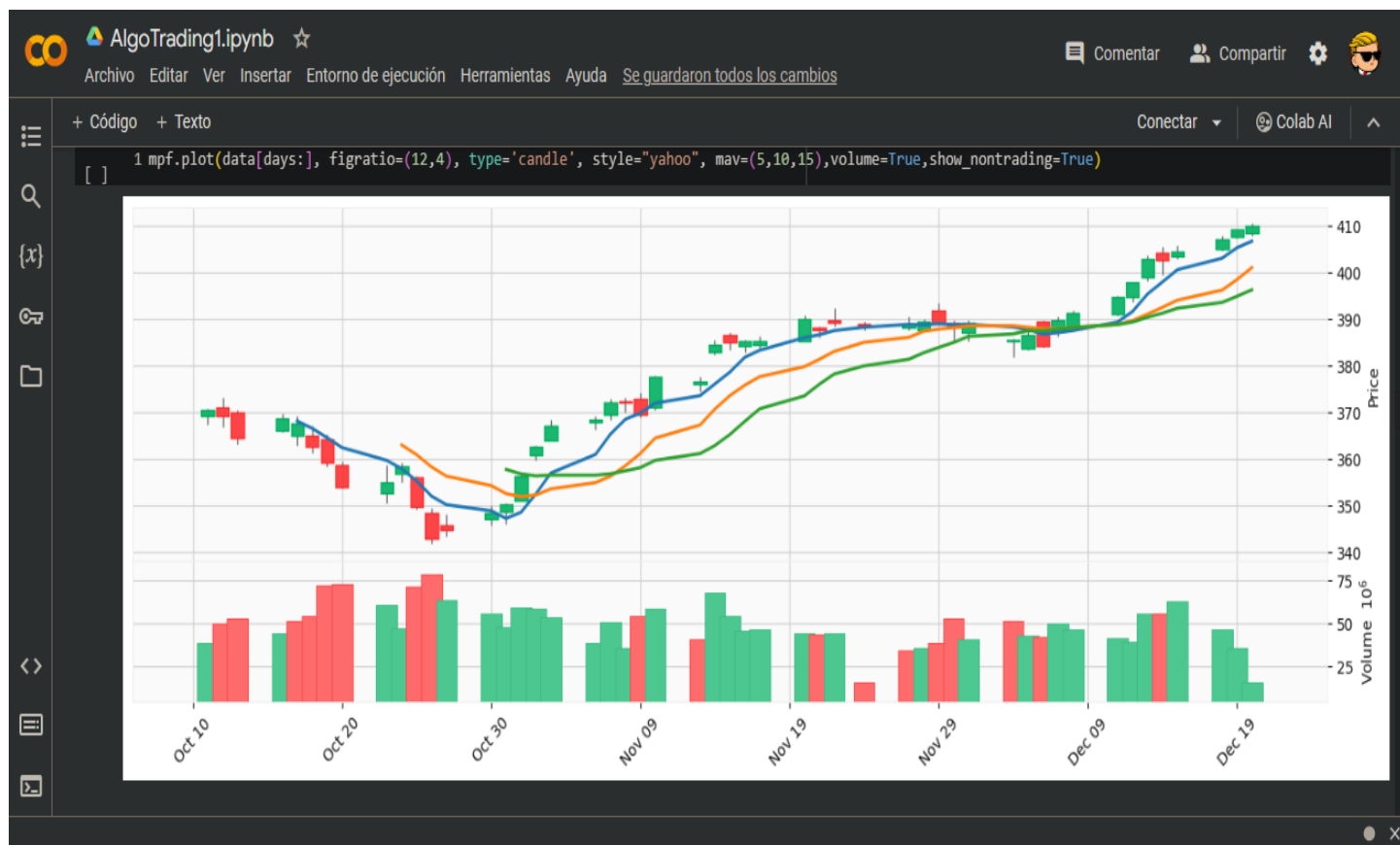


We include Volume



The above chart excluding non-trading days like Saturday, Sunday, or holidays. Sometimes it is helpful to include non-trading days. Candle chart with non-trading days

Suppose we want to show the non-trading day:



Popular technical indicators

Technical indicators are mathematical formulas used in technical analysis. They are based on historical price data and are used to identify patterns and trends in the market. Technical indicators are lagging indicators. Strictly speaking they cannot forecast future price movements. But some price trends can be foreseen.

Technical indicators can help to get insights into future price movements. Traders use them to form buy and sell strategies, to find the key levels of support and resistance, and to set stop loss levels.

One of the most popular open source Python library for technical analysis is ta-lib (Technical Analysis Library). Let's get to know this library, and then let's learn a few common technical indicators introduce Moving Averages (MA), Relative Strength Index (RSI), and Bollinger Bands (BB).

The TA-Lib library has more than 150 technical indicators. It is widely used by software developers and quants to perform technical analysis.

co

AlgoTrading1.ipynb ☆

ComentarCompartir⚙️🧑

ArchivoEditarVerInsertarEntorno de ejecuciónHerramientasAyudaSe guardaron todos los cambios

+ Código+ Texto

Conectar ▾Colab AI ^

Simple Moving Averages (SMA)

This indicator calculates the average price of a security over a specific time period. I generate a 5-day moving average and a 20-day moving average. The shorter term moving average is called the fast moving average and the longer term moving average is called the slow moving average. The syntax is talib.SMA(price, timeperiod).

```
[ ] 1 import talib
2 data['SMAS'] = talib.SMA(data['Close'], timeperiod=5)
3 data['SMA20'] = talib.SMA(data['Close'], timeperiod=20)
4 #data.dropna(inplace=True)
5 data
```

	Date	Open	High	Low	Close	Volume	Market Cap	Time Open	Time High	Time Low	Time
0	2023-12-19	42641.510213	43354.296597	41826.335152	42270.526140	2.317100e+10	8.274471e+11	2023-12-19T00:00:00.000Z	2023-12-19T02:01:00.000Z	2023-12-19T17:59:00.000Z	2023-12-19T23:59:59.999Z
1	2023-12-18	41348.202089	42720.297127	40530.258519	42623.538325	2.522464e+10	8.343121e+11	2023-12-18T00:00:00.000Z	2023-12-18T02:14:00.000Z	2023-12-18T10:38:00.000Z	2023-12-18T23:59:59.999Z
2	2023-12-17	42236.108345	42359.496119	41274.543518	41364.665918	1.667870e+10	8.096285e+11	2023-12-17T00:00:00.000Z	2023-12-17T02:17:15.000Z	2023-12-17T17:38:00.000Z	2023-12-17T23:59:59.999Z
3	2023-12-16	41937.740968	42664.945136	41723.113466	42240.119082	1.438673e+10	8.267228e+11	2023-12-16T00:00:00.000Z	2023-12-16T02:16:45.000Z	2023-12-16T00:40:00.000Z	2023-12-16T23:59:59.999Z
...	...	...	...	...	...	...	...	...	...	...	...

co

AlgoTrading1.ipynb ☆

ComentarCompartir⚙️🧑

ArchivoEditarVerInsertarEntorno de ejecuciónHerramientasAyudaSe guardaron todos los cambios

+ Código+ Texto

Conectar ▾Colab AI ^

[]

2	2023-12-17	42236.108345	42359.496119	41274.543518	41364.665918	1.667870e+10	8.096285e+11	2023-12-17T00:00:00.000Z	2023-12-17T02:17:15.000Z	2023-12-17T17:38:00.000Z	2023-12-17T23:59:59.999Z
3	2023-12-16	41937.740968	42664.945136	41723.113466	42240.119082	1.438673e+10	8.267228e+11	2023-12-16T00:00:00.000Z	2023-12-16T02:16:45.000Z	2023-12-16T00:40:00.000Z	2023-12-16T23:59:59.999Z
4	2023-12-15	43028.250691	43087.823773	41692.968038	41929.758393	1.963944e+10	8.206109e+11	2023-12-15T00:00:00.000Z	2023-12-15T02:00:30.000Z	2023-12-15T15:47:00.000Z	2023-12-15T23:59:59.999Z
...	...	...	...	...	...	...	...	...	...	...	...
3883	2013-05-02	116.379997	125.599998	92.281898	105.209999	0.000000e+00	1.168517e+09	2013-05-02T00:00:00.000Z	2013-05-02T14:25:01.000Z	2013-05-02T14:30:02.000Z	2013-05-02T23:59:59.999Z
3884	2013-05-01	139.000000	139.889999	107.720001	116.989998	0.000000e+00	1.298955e+09	2013-05-01T00:00:00.000Z	2013-05-01T00:15:01.000Z	2013-05-01T19:55:01.000Z	2013-05-01T23:59:59.999Z
3885	2013-04-30	144.000000	146.929993	134.050003	139.000000	0.000000e+00	1.542813e+09	2013-04-30T00:00:00.000Z	2013-04-30T08:25:02.000Z	2013-04-30T18:55:01.000Z	2013-04-30T23:59:59.999Z
3886	2013-04-29	134.444000	147.488007	134.000000	144.539993	0.000000e+00	1.603769e+09	2013-04-29T00:00:00.000Z	2013-04-29T13:15:01.000Z	2013-04-29T05:20:01.000Z	2013-04-29T23:59:59.999Z
3887	2013-04-28	136.689455	143.365942	110.845593	127.954255	2.595684e+04	1.419008e+09	2013-04-28T00:00:00.000Z	2013-04-28T00:45:00.000Z	2013-04-28T13:05:00.000Z	2013-04-28T23:59:59.999Z

3888 rows × 15 columns


AlgoTrading1.ipynb
☆

Archivo
Editar
Ver
Insertar
Entorno de ejecución
Herramientas
Ayuda
Se guardaron todos los cambios

Comentar
Compartir
⚙️


+ Código
+ Texto
Conectar
Colab AI
⬆️

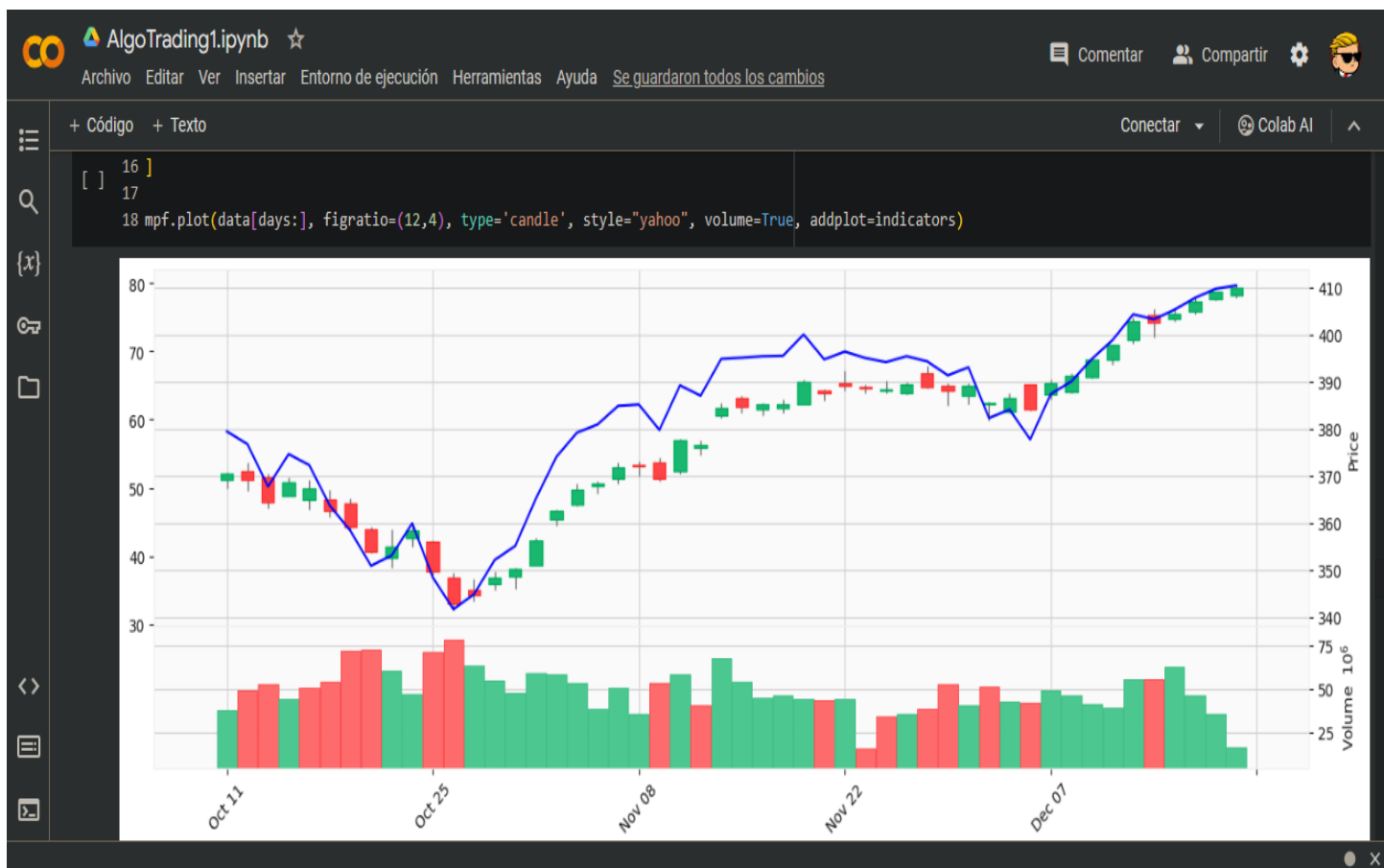
Relative Strength Index (RSI)

Sometimes we want to understand if a stock is overbought or oversold. If it is oversold, the price can be too low that triggers a buy signal. And if it is overbought, the price can be too high and we should prepare to exit. The RSI indicator measures the magnitude of recent price changes to determine overbought or oversold conditions. It is calculated as the ratio of average gains to average losses over a specific time period. Let's calculate RSI and plot it as a blue line.

```

[ ] 1 # Retrieve historical stock price data for QQQ
2 qq = yf.Ticker("QQQ")
3 data = qq.history(start="2021-01-01", end="2023-12-31")
4
5 RSI_PERIOD = 14
6 data['rsi'] = talib.RSI(data['Close'], RSI_PERIOD)
7 data.tail(10)
8
9 # Plot it
10 import mplfinance as mpf
11
12 days = -50
13
14 indicators = [
15     mpf.make_addplot(data['rsi'][days:], color='blue', width=1.5),
16 ]
17
18 mpf.plot(data[days:], figratio=(12,4), type='candle', style="yahoo", volume=True, addplot=indicators)

```



RSI is a single line. When it is above a certain level, typically 70, it signals an overbought condition, so it is a sell signal. When it is below a level, typically 30, it indicates an oversold condition so is a buy signal.

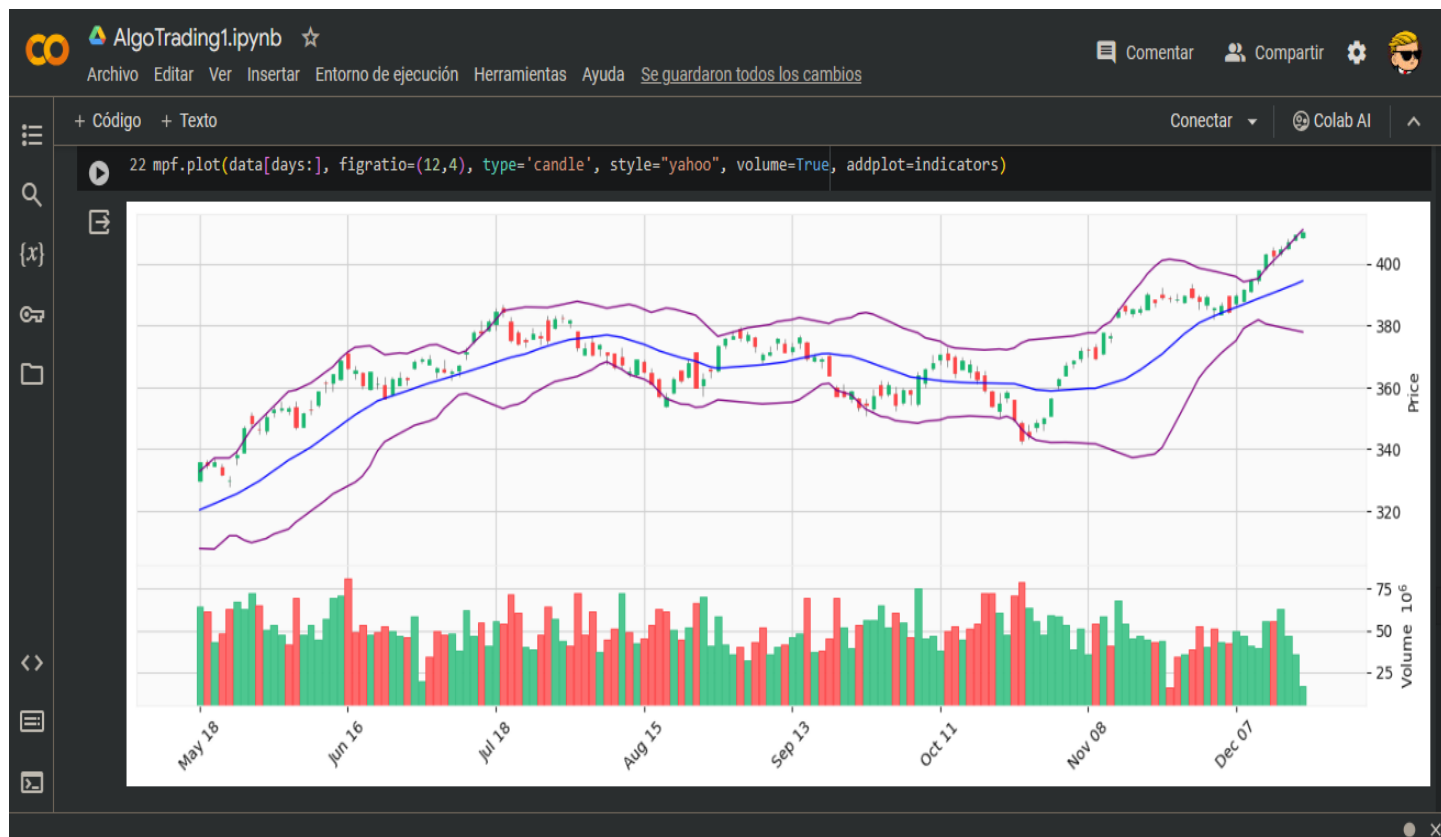
Bollinger Bands (BBANDS)

Bollinger Bands plot two standard deviations around a moving average to provide a visual representation of volatility. It can be used to identify breakouts and potential reversals in the market. The gap between the upper band and the lower band is called the Bollinger Bands.

It generates buy signals when the price falls below the lower band, indicating a potential oversold condition, and sell signals when the price rises above the upper band, indicating a potential overbought condition.

```
AlgoTrading1.ipynb ☆
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios
+ Código + Texto Conectar Colab AI
rises above the upper band, indicating a potential overbought condition.

[ ] 1 # Retrieve historical stock price data for QQQ
    2 qqq = yf.Ticker("QQQ")
    3 data = qqq.history(start="2021-01-01", end="2023-12-31")
    4
    5 # Define the Bollinger Bands with 2-sd
    6 data['upper_2sd'], data['mid_2sd'], data['lower_2sd'] = talib.BBANDS(data['Close'],
    7                                                                    nbdevup=2,
    8                                                                    nbdevdn=2,
    9                                                                    timeperiod=20)
    10
    11 # Plot
    12 import mplfinance as mpf
    13
    14 days = -150
    15
    16 indicators = [
    17     mpf.make_addplot(data['upper_2sd'][days:], color='purple', width=1),
    18     mpf.make_addplot(data['mid_2sd'][days:], color='blue', width=1),
    19     mpf.make_addplot(data['lower_2sd'][days:], color='purple', width=1)
    20 ]
    21
    22 mpf.plot(data[days:], figratio=(12,4), type='candle', style="yahoo", volume=True, addplot=indicators)
```



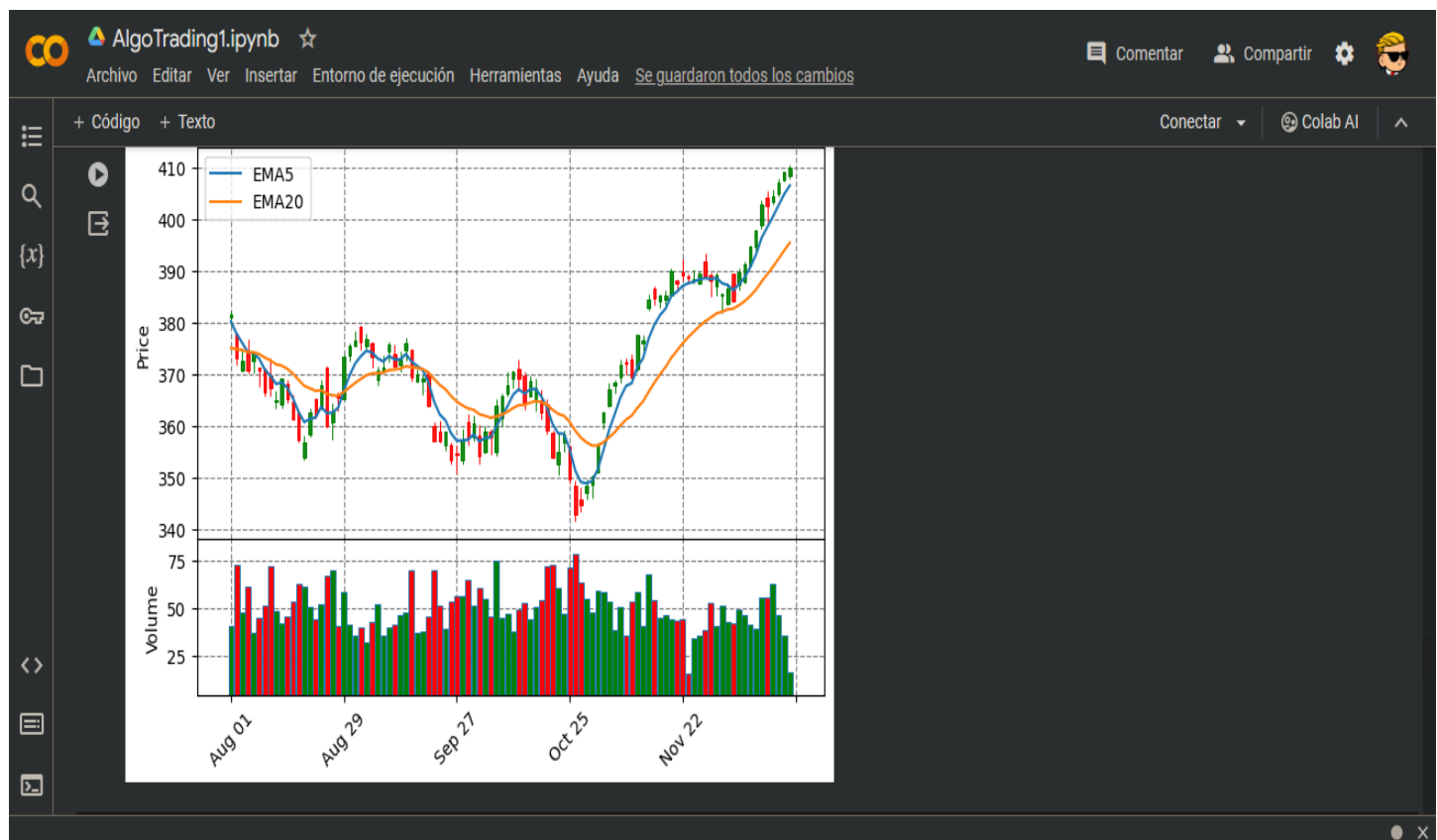
It generates the following graph. You can observe there are times when the price touching the upper band, indicating a sell signal. Or the price touches the lower band, indicating a buy signal.

All other technical indicators in TA-lib

There are around 158 functions in TA-lib. You can print them out by running the following code.

```
AlgoTrading1.ipynb
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda Se guardaron todos los cambios
+ Código + Texto
Conectar Colab AI
At the time I am writing this post, there are 158 functions in TA-lib. You can print them out by running the following code.

[ ] 1 data['EMA5'] = talib.EMA(data['Close'], timeperiod=5)
    2 data['EMA20'] = talib.EMA(data['Close'], timeperiod=20)
    3
    4 import mplfinance as mpf
    5 import matplotlib.pyplot as plt
    6 style = mpf.make_mpf_style(marketcolors=mpf.make_marketcolors(up="g", down="r",inherit=True),
    7                           gridcolor="gray", gridstyle="--", gridaxis="both")
    8 days = -100
    9
   10 added_plots = {"EMA5" : mpf.make_addplot(data['EMA5'].iloc[days:],width=1.5),
   11               "EMA20" : mpf.make_addplot(data['EMA20'].iloc[days:],width=1.5)}
   12
   13 plt.figure(figsize=(10,20))
   14 fig, axes = mpf.plot(data.iloc[days:, 0:5], type="candle", style=style,
   15                    addplot=list(added_plots.values()),
   16                    volume=True,
   17                    returnfig=True)
   18
   19 axes[0].legend([None]*(len(added_plots)+2))
   20 handles = axes[0].get_legend().legend_handles
   21 axes[0].legend(handles=handles[2:],labels=list(added_plots.keys()))
   22
   23 axes[0].set_ylabel("Price")
   24 axes[1].set_ylabel("Volume")
```




AlgoTrading1.ipynb
☆

Archivo
Editar
Ver
Insertar
Entorno de ejecución
Herramientas
Ayuda
Se guardaron todos los cambios

Comentar
Compartir
⚙️


+ Código
+ Texto
Conectar
Colab AI
^

```
[ ] 1 import talib
    2 # dict of functions by group
    3 ta_func_grp = talib.get_function_groups()
    4 ta_func_grp

{'Cycle Indicators': ['HT_DCPERIOD',
                     'HT_DCPHASE',
                     'HT_PHASOR',
                     'HT_SINE',
                     'HT_TRENDMODE'],
 'Math Operators': ['ADD',
                    'DIV',
                    'MAX',
                    'MAXINDEX',
                    'MIN',
                    'MININDEX',
                    'MINMAX',
                    'MINMAXINDEX',
                    'MULT',
                    'SUB',
                    'SUM'],
 'Math Transform': ['ACOS',
                    'ASIN',
                    'ATAN',
                    'CEIL',
                    'COS',
                    'COSH',
```


AlgoTrading1.ipynb
☆

Archivo
Editar
Ver
Insertar
Entorno de ejecución
Herramientas
Ayuda
Se guardaron todos los cambios

Comentar
Compartir
⚙️


+ Código
+ Texto
Conectar
Colab AI
^

You can count the number of functions by group:

```
[ ] 1 for key, value in ta_func_grp.items():
    2     print(key, ":", len([item for item in value if item]))

Cycle Indicators : 5
Math Operators : 11
Math Transform : 15
Momentum Indicators : 30
Overlap Studies : 17
Pattern Recognition : 61
Price Transform : 4
Statistic Functions : 9
Volatility Indicators : 3
Volume Indicators : 3
```

↑ ↓ ↻ ⌨️ 📄 🗑️ ⋮

ynfinance only offers tick data for 7 days. If you need historical tick data for a long period, you will need to order from tick data providers.

Free download for the cryptocurrency data. Other than stock market data, here let's learn how to download cryptocurrency data. The cryptocmd library offers free historical cryptocurrency data from CoinMarketCap. As of today, the top five cryptocurrencies are Bitcoin ("BTC"), Ethereum ("ETH"), Tether USD ("USDT"), Ripple ("XRP"), and Binance Coin "BNB". Below I download the data for "XRP". All these data downloads have the same data format like "Date", "Open", "High", "Low", "Close", "Volume". This enables easy programming.

CO

AlgoTrading1.ipynb ☆

ComentarCompartirColab AI

ArchivoEditarVerInsertarEntorno de ejecuciónHerramientasAyudaSe guardaron todos los cambios

+ Código + Texto

ConectarColab AI

same data format like "Date", "Open", "High", "Low", "Close", "Volume". This enables easy programming.

```
1 from cryptocmd import CmcScraper
2
3 # initialise scraper without time interval
4 scraper = CmcScraper("BTC")
5
6 # Pandas dataframe for the same data
7 data = scraper.get_dataframe()
8 data
```

Requirement already satisfied: cryptocmd in /usr/local/lib/python3.10/dist-packages (0.6.4)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from cryptocmd) (2.31.0)
Requirement already satisfied: tablib in /usr/local/lib/python3.10/dist-packages (from cryptocmd) (3.5.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->cryptocmd) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->cryptocmd) (3.6)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->cryptocmd) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->cryptocmd) (2023.11.17)

	Date	Open	High	Low	Close	Volume	Market Cap	Time Open	Time High	Time Low	Time
0	2023-12-19	42641.510213	43354.296597	41826.335152	42270.526140	2.317100e+10	8.274471e+11	2023-12-19T00:00:00.000Z	2023-12-19T02:01:00.000Z	2023-12-19T17:59:00.000Z	2023-12-19T23:59:59.999Z
1	2023-12-18	41348.202089	42720.297127	40530.258519	42623.538325	2.522464e+10	8.343121e+11	2023-12-18T00:00:00.000Z	2023-12-18T21:49:00.000Z	2023-12-18T10:38:00.000Z	2023-12-18T23:59:59.999Z

CO

AlgoTrading1.ipynb ☆

ComentarCompartirColab AI

ArchivoEditarVerInsertarEntorno de ejecuciónHerramientasAyudaSe guardaron todos los cambios

+ Código + Texto

ConectarColab AI

```
2 2023-12-17 42236.108345 42359.496119 41274.543518 41364.665918 1.667870e+10 8.096285e+11 2023-12-17T00:00:00.000Z 2023-12-17T17:15:00.000Z 2023-12-17T17:15:00.000Z 2023-12-17T23:59:59.999Z
3 2023-12-16 41937.740968 42664.945136 41723.113466 42240.119082 1.438673e+10 8.267228e+11 2023-12-16T00:00:00.000Z 2023-12-16T16:45:00.000Z 2023-12-16T00:40:00.000Z 2023-12-16T23:59:59.999Z
4 2023-12-15 43028.250691 43087.823773 41692.968038 41929.758393 1.963944e+10 8.206109e+11 2023-12-15T00:00:00.000Z 2023-12-15T00:30:00.000Z 2023-12-15T15:47:00.000Z 2023-12-15T23:59:59.999Z
... ..
3883 2013-05-02 116.379997 125.599998 92.281898 105.209999 0.000000e+00 1.168517e+09 2013-05-02T00:00:00.000Z 2013-05-02T14:25:01.000Z 2013-05-02T14:30:02.000Z 2013-05-02T23:59:59.999Z
3884 2013-05-01 139.000000 139.889999 107.720001 116.989998 0.000000e+00 1.298955e+09 2013-05-01T00:00:00.000Z 2013-05-01T00:15:01.000Z 2013-05-01T19:55:01.000Z 2013-05-01T23:59:59.999Z
3885 2013-04-30 144.000000 146.929993 134.050003 139.000000 0.000000e+00 1.542813e+09 2013-04-30T00:00:00.000Z 2013-04-30T08:25:02.000Z 2013-04-30T18:55:01.000Z 2013-04-30T23:59:59.999Z
3886 2013-04-29 134.444000 147.488007 134.000000 144.539993 0.000000e+00 1.603769e+09 2013-04-29T00:00:00.000Z 2013-04-29T13:15:01.000Z 2013-04-29T05:20:01.000Z 2013-04-29T23:59:59.999Z
3887 2013-04-28 136.689455 143.365942 110.845593 127.954255 2.595684e+04 1.419008e+09 2013-04-28T00:00:00.000Z 2013-04-28T00:45:00.000Z 2013-04-28T13:05:00.000Z 2013-04-28T23:59:59.999Z
```

3888 rows x 11 columns

